Access Control

OSTEP 55

Sam Ogden

Updated 2026-05-13 11:07

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

The Big Question

How do we control who gets what resources?

Background on Access Controls

Key definitions

1. **Subject**: The entity that wants to perform the access
    - A process representing a user
2. **Object**: The thing the *subject* wants to access
    - e.g. a file
3. **Access**: The mode of dealing with the *object*
    - e.g. writing to a file

Authorization: Determining whether a *subject* can *access* an *object*

Authorization

Before allowing access to a resource, we need to do authorization.

Our **reference monitor** is what performs this

Resources might be handled in different ways

- Memory
  - Virtual memory
  - Page sharing?
- Files
  - Executable programs
  - log files
- Devices
  - A keyboard

Accessing Resources

A good way to think about how we are opening resources is to think of opening a file.

```
1 open("/var/foo", O_RDWR)
```

`open(...)` is a syscall

When access to the resource is requested the OS sees if the subject is allowed to access it.

Access Control Lists and Capabilities

Access Control Lists

- List of who can access resources
- Attached to the resources themselves

Capabilities

- A token that grants access
- Held when access is requested

Access Control Lists

Access Control Lists

- Core idea is that we check users against a list of who is allowed to access the resource
- If the user is on the list, they can access the resource
- If *not* then the access is rejected

How do ACLs work?

1. When a resource is requested the OS finds the ACL associated with it
2. The OS searches through the list for the current *subject*
    - For a process, this is the user that owns the process
3. If the *subject* is on the list ⇒ *Allow*
    - Otherwise ⇒ *Disallow*

Are they good?

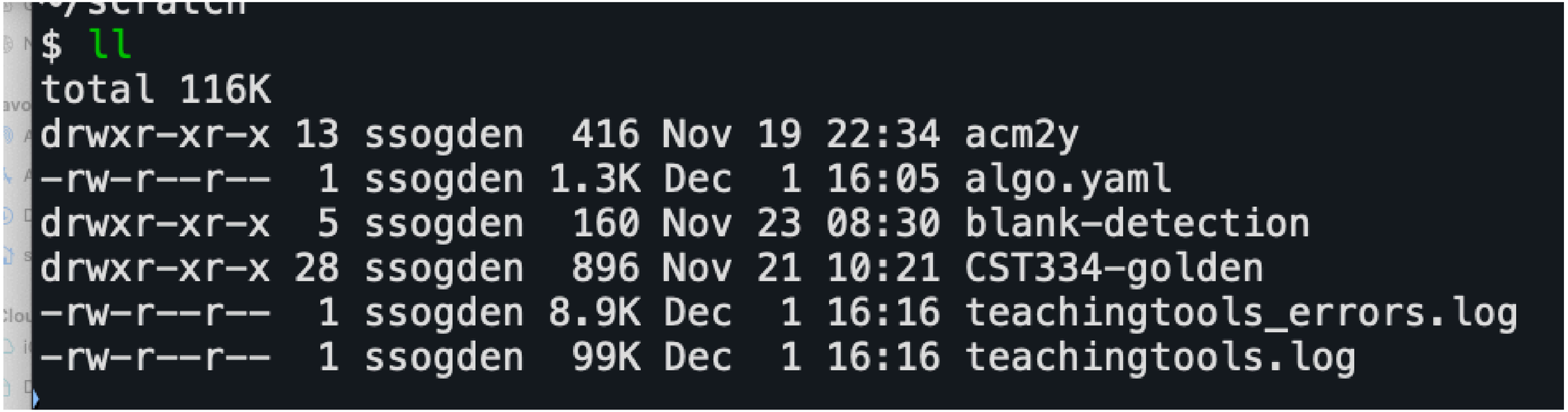
Pros

- Easy to check
- Easy to change
- Close to the resource typically

Cons

- Where do we store it?
- What if we want to know all resources a *subject* can access?
- How do we identify users across networks?
- We have to check each time we access a resource

What do ACLs look like?



ACLs are the basis in Linux

Capabilities

Capabilities

- Core idea is that *subjects* have a token that identifies them
  - Such as a sign saying they aren't Elon
- If the subject doesn't have the token, they aren't allowed

How do capabilities work?

1. When the *subject* requests access the OS we check if the process has that capability
2. If the process has that capability then we allow it
    - If not we deny

Okay, but how do they actually work?

This sounds like ACLs so far, with the lookup...

The capabilities list isn't a "file"; it's assigned to the process.

It'll be kept in the Process Control struct (like the context!)

What are capabilities?

Capabilities are *just* a binary string

If it’s short then a process can guess it!

If it’s long and cryptographic then we’re probably safe!

Capabilities pros and cons

Pros

- It’s easy to check
- Revoking access is super simple
- It’s cheap to check!

Cons

- Checking everybody who can access a resource is painful
- There’s been less work on making them work well

Which is better?

As always, it depends...

Linux uses a combination of both

- When you are trying to access a file, the ACL is queried
- When you have a file open, you have a file handle
  - The file handle *is* a capability

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
  char *mem;                // Start of process memory
  uint sz;                  // Size of process memory
  char *kstack;             // Bottom of kernel stack
                            // for this process
  enum proc_state state;    // Process state
  int pid;                  // Process ID
  struct proc *parent;      // Parent process
  void *chan;               // If !zero, sleeping on chan
  int killed;               // If !zero, has been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;        // Current directory
  struct context context;   // Switch here to run process
  struct trapframe *tf;     // Trap frame for the
                            // current interrupt
};
```

Role-Based Access Control (RBAC)

Core Idea:

Let’s assign access based on people’s roles

Example:

Programmers shouldn’t touch payroll.

Accounting shouldn’t touch code.

We can switch between groups depending on what role we are currently in.

This is an example of *least privilege*

Who decides this?

Mandatory vs Discretionary

Mandatory	Discretionary
A system admin decides who gets access	Owner of the file decides
Often used in controlled environments	The default on most systems

Dangerous Things

How to do dangerous things

Sometimes it’s necessary to run applications with higher privileges than you have.

For example, parts of the windowing / display stack may need extra permissions (hardware, input devices, etc.)

`setuid`

This allows us to run a program as the *owner* of the file rather than as ourselves

`setuid`

Uses

- Allows us to run programs with escalated privileges
  - Run as root!
- Allows us to run as somebody else
  - CRON jobs!

Dangers

- That program had better be safe!

A concrete access-control mess: X11 / Xorg

The **X11 protocol** (and X servers that implement it, like **Xorg**) are a great example of how “access control” shows up in real systems.

- If a program can connect to your X server, it can often:
  - Read keystrokes / clipboard
  - Take screenshots
  - Inject mouse + keyboard events

So: **“Who is allowed to connect to my display?”** is an access-control question.

X11 uses both ACL-ish and capability-ish ideas

- The X server maintains an “allowed clients” policy (ACL flavor)
- But the common mechanism is a secret token:
  - Your X session cookie in `~/.Xauthority`
  - If someone gets it, they can often connect (capability flavor)

This is why accidentally sharing credentials (or disabling checks) is such a big deal.

Why `setuid` is scary here

- Historically, the **Xorg X server** sometimes needed elevated privileges on some systems
  - So you might see `Xorg` (or a wrapper like `Xorg.wrap`) installed `setuid root`
- Any memory-safety bug in a `setuid` binary can become a **privilege escalation**

Design goal: keep privileged code **small**, and drop privileges as soon as possible.

How Xorg reduced its `setuid` surface (rootless X)

Modern Linux setups try to keep Xorg unprivileged and move the dangerous parts elsewhere.

- A small privileged helper / service (e.g., `systemd-logind`) enforces policy:
  - "Is this user/session allowed to use the GPU / input devices?"
  - Often implemented via device ACLs tied to the active seat/session
- The helper opens device files (like `/dev/dri/*`) and passes **file descriptors** to Xorg
  - Those FDs are effectively **capabilities**: if you have the handle, you can use the device

Net effect: privileged code is smaller, and Xorg can drop (or avoid) `setuid root`.

Conclusion

- Access control is **authorization**: can a subject do an action to an object?
- ACLs and capabilities are two useful mental models (and real systems mix them)
- Least privilege (RBAC, groups) helps reduce blast radius
- `setuid` is a sharp tool: necessary sometimes, dangerous always
- Concrete reminder: X11/Xorg is “just access control” in disguise (and mistakes are painful)